

Empirical and Analytic Discovery in IL

MICHAEL H. SIMS

(SIMS@AMES-PLUTO.ARPA)

*Departments of Computer Science and Mathematics, Rutgers University, New Brunswick, NJ 08903 U.S.A., and
AI Research and Applications Branch, NASA Ames, Moffett Field, Ca. 94035 U.S.A.*

Abstract

The discovery process in many domains entails a rich interaction of reasoning techniques - such as observing patterns, reasoning about examples, suggesting experiments, evaluating results, and formal proofs based on current models. The goal of my research is to investigate the synergistic interactions of these reasoning techniques leading to discovery. This investigation has involved my developing an agenda based discovery system, called IL¹, with which I have investigated a sub-domain of mathematics, Conway Numbers. Although IL does AM-like exploration of its domain (Davis & Lenat, 1982), IL is distinct in a number of ways, including the representations used and IL's use of theorem proving and explanation based learning (EBL). In this paper we discuss the control structure of IL and a particular test case which illustrates the interplay of examples with deduction. IL's theorem prover, VERIFY, is used for explanation based learning as well as for the verification of properties.

1 Introduction

Discovery is a ubiquitous human activity that has been used as a metaphor to describe the activities of several AI systems. This paper is concerned with a particular synergy which can occur during the discovery process in many formal domains when the investigation of examples (empirical techniques) interplays with deductions (analytical techniques). This interplay is the crux of the scientific method as it is used in formal domains, such as mathematics. This process can be abstracted as a four step cycle:

- observe particular instance of some property
- conjecture that property holds generally
- support conjecture with further experiments/examples

¹IL is named for Imre Lakatos (Lakatos, 1976).

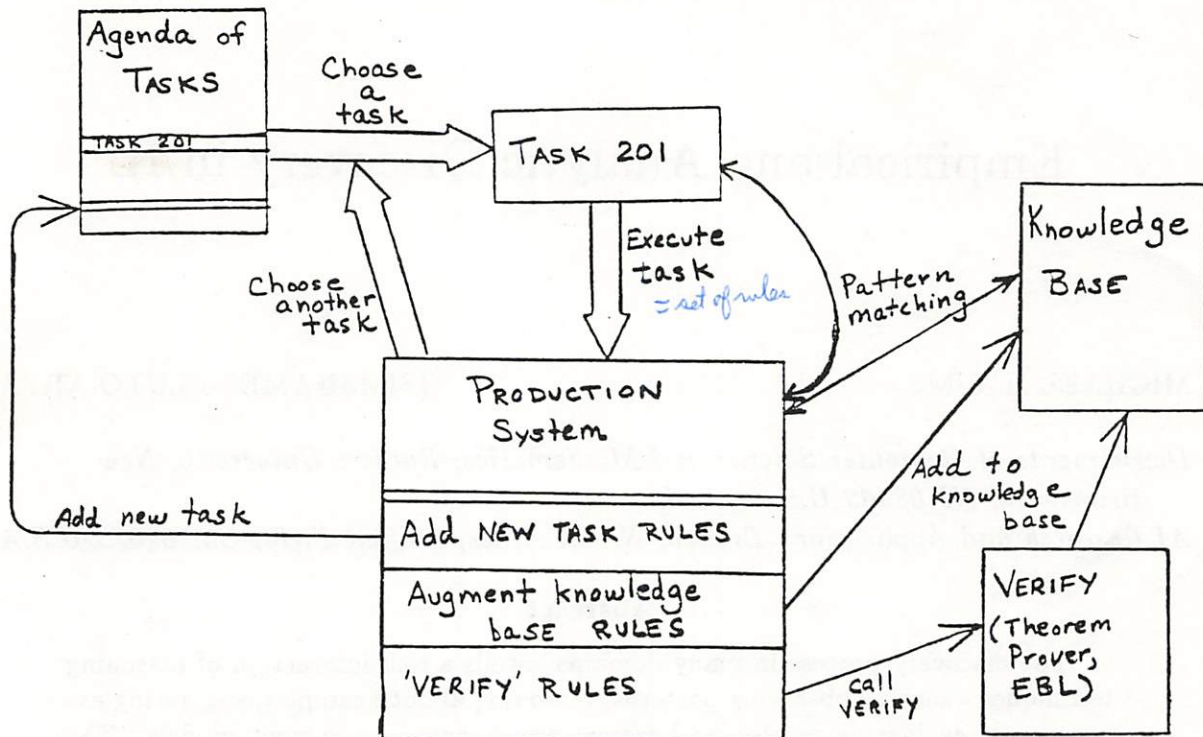


Figure 1: Control structure of IL

- prove property

My goal in this reported research is to model that synergistic coupling of analytic and deductive techniques - not necessarily in a cognitively accurate way. I will first describe the control structure of IL, then illustrate the above four step cycle with a particular example.

2 IL

At the top level of control IL has an agenda of tasks. One task is selected and executed, and then this cycle is repeated. A task is executed by a production system which uses that task as part of its working memory to be pattern matched against. The production system cycles through a set of rules until no more are applicable and then another task is chosen.

2.1 Task selection

Tasks on the agenda are ordered by worth - a measure associated with each task. The worths are simply a dynamically changeable, numeric value associated with each concept (tasks are special kinds of concepts). The worth values are intended to reflect the interestingness of a given concept.

The method for choosing a task to execute is to pick a random task from among the n tasks with the highest worth. Two justifications can be given for this introduction of randomness. First, there may not be any important significance to minor numeral distinctions between the top few tasks. Second, the objective of IL is to do *exploration* of its domain, and the randomness allows for the exploration of *different* parts of its exploration domain on successive runs. good

The worth for a *task* can be influenced by the users interest, issues which are considered fundamental for the particular domain (such as the classification of groups in group theory), recent tasks, the concepts recently investigated, the reasons given for suggesting the task, and heuristically suggested interesting relations between concepts.

2.2 Task execution: the production system

The production system portion of IL pattern matches against the current task and the current knowledge base to decide which rules² are applicable. Conflict resolution among a competing set of rules is done by a dynamically modifiable, rule weighting and by a simple timing scheme which classifies rules as pre-task, task, and post-task. The production system cycles until there are no more applicable rules, at which time another task is chosen.

The rules are of three types: those that add new tasks to the agenda, those that add new knowledge to the knowledge base, and rules which call VERIFY, the theorem prover (this is really a special case of the second type but worth describing separately).

Rules of the first type may suggest sub-tasks of a current task or new tasks. These rules may also suggest new reasons why an old task is a good thing to do, and this may effect the task's worth and hence its chance of being chosen.

Rules of the second type may augment the knowledge base by doing such things as noticing relations between existing concepts, changing concept's worth, and generating possible examples for a given concept.

Rules of the third type call the VERIFY theorem prover. The VERIFY module does heuristically controlled, depth limited, depth first search for proofs of expressions and generates a trace of its activities which is readable by the rest of IL. It also uses domain specific information to decide upon proof strategy, and the caching of proofs, theorems and intermediate results. By a general use of VERIFY's heuristic control mechanisms I am able to utilize the general theorem prover to do explanation based learning, and thereby replace IL's previous EBL system, IL-BP, as described in (Sims, 1986). It is also used in the verification of examples and the verification of rule preconditions.

IL's concepts are organized into an hierarchy of frames, which contain the domain knowledge and much of the control information. This supporting frame system, called PARL, allows a frame organization and flexible mechanisms for inheritance and slot retrieval within a Prolog environment.

²What we are calling rules, generally Lenat has called heuristics. In both cases the objects are heuristic in nature, and we call them here *rules* to emphasis the connection with rule based systems.

2.3 Representations

One of the underlying principles of IL is that representations of the objects be as close as possible to those used by mathematicians. This is true for the concepts used, the theorems and proofs, as well as the heuristic rules shown above. For example, let's consider the theorem that if A is an empty set, then for no X which is an element of A is Y true, where Y is arbitrary. In IL I represent this as:

```
Theorem name: t108
slots:
  generalizations: [theorem]
  theorem:
    [implies,
      [[empty_set, [A]],
       [for_no, [X,A,Y]]]
    ]
...

```

Such declarative representations have utility only if there are underlying methods for interpreting these facts. In IL there are a number of interpreters (currently about half a dozen) which interpret these declarative domain elements. There are also a number of low level Prolog functions which implement the most primitive concepts. For example, there is a primitive routine which determines whether a set is empty or not. Inside of IL these low level routines fall into two classes - primitive and simple.

In order to explain the distinction between simple and primitive predicates it is useful to recall the 'closed world assumption' of Prolog, which implies that whatever cannot be proved is assumed to be false. This is not how mathematical reasoning works, and any reasonable theorem prover (eg., VERIFY) must distinguish between what cannot be proved and what can be proved false. On the other hand, there are some objects which can be modeled in Prolog whose truth or falseness in the Prolog sense corresponds exactly to their truth or falseness in the logical sense. These are particularly simple to implement, and we say that the concept definitions are *simple*. All the other low level objects in IL are called *primitive*. Together the simple and primitive objects make up the low level foundations out of which all other IL objects are defined.

These foundational concepts correspond to undefined mathematical concepts, and are not accessible to IL's reasoning. Just as it is not possible to do mathematics without undefined objects, it is not possible for us to do computer based reasoning without these undefined objects. They in effect form the ultimate bias of our reasoning system. Even though these objects will remain implicit to IL's reasoning, the strength of IL and its interpreters is that it can build objects out of these IL-undefined objects which are *explicitly* represented (see Keller, 1987). It is the one additional layer of interpreters (beyond what is present in AM and Eurisko) which allows us to use these more explicit and declarative representations for the mathematical concepts. This not only eases the knowledge engineering task, but also allows IL to maintain a close match between its

concepts and their representation, and support the coupling of empirical and analytical elements.

3 An Example

In order to be clear about the sort of reasoning that is being modeling in this paper, let's consider a specific example in more detail. The domain for demonstration for this general method is a field of mathematics called Conway or Surreal Numbers (Conway, 1976). For our present purposes it is sufficient to understand Conway Numbers to be a class which can be shown to contain a set which is isomorphic to the real numbers. It also contains objects which correspond to infinite and infinitesimal numbers. Although we know that the Conway Numbers contain the reals, one can begin IL without it knowing that fact. Currently I am investigating the discovery of that property by IL.

In what follows I will describe the process by which IL could discover the property of additive identity and subsequently verify it. Since IL is an heuristically driven agenda system, its activity at any given point is dynamically decided with induced randomness and IL might pursue many different avenues of investigation. In a sense, IL is doing exploration of its domain, as opposed to trying to solve any specific problem.

For clarity, I will here represent the subset of Conway Numbers which correspond to the real numbers by the usual notation for numbers (eg., -1,0,1), even though IL's standard representation of these objects is quite different. For IL, the numbers 0,1, etc. and the operators +, >, etc. have definitions which look quite different from those usually associated with these numbers and operators. These definitions were first given by John Conway.

What is indicated in this paper is a scenario whereby IL discovers that there is a number, e (actually it will be 0), which when added to any other number gives back that number. i.e., $X + e = X$, and $e + X = X$. This element e is called an additive identity element. IL's verification that the Conway numbers contain an additive identity is one step in its proving that Conway numbers contain the reals.

This example will follow the four step discovery process mentioned above. The following are tasks and rules that IL would use to carry this out. They are listed here in the order which they might be executed. The tasks are goals to be accomplished, and the rules are suggestions about control and activities.

- Task 1: Generate examples of Conway addition, +.
- Rule 1: If there are few examples of the domain of operator (here +) then suggest sub-task (i.e., place this task on the agenda of possible tasks to be executed) to generate examples of domain. Suggested task: generate examples of Conway Numbers.
- Task 2: Generate examples of Conway Numbers. {Several rules would go to work here, with a result of the generation of some Conway Numbers, say -2,-1,0,1,2.}

- Rule 2: Since addition of Conway Numbers is known to be an interesting concept, suggest sub-task of trying to find examples of properties for addition.
- Task 3: Find examples of properties for addition.
- Rule 3: If task is to find examples of properties for X, then check if examples of X satisfy any known property. In this case let's say that the result is that, *identity is 0*, is satisfied by all known examples. Enter 'identity element is 0' as a possible property for addition. The proof of any one of the properties, say $0+1=1$, usually requires the task of calling IL's theorem prover.
- Rule 4: If task is to find examples of properties for X, and there is an entry in the 'possible property' slot, then suggest tasks: (a) generate more examples of X (here addition), (b) {execute after (a)} check that those new examples also satisfy the additive identity property, and (c) {execute after (a) and (b)} prove that 0 is an additive identity for Conway numbers. Suggested proof method: first try this proof of additive identity property by generalizing a proof of a particular instance (i.e., use IL's EBG capabilities).
- Task 4: Generate *more* examples of addition.
- Task 5 Check that these new examples of addition satisfy possible property of 0 being an additive identity.
- Task 6: Prove 0 is the additive identity.
- Rule 5: Try proving 0 is additive identity by generalizing proof of specific known example. i.e., first try proving by using EBG techniques on a particular example, say $0+1=1$, to guide the proof of general case.
- Rule 6: If EBG doesn't work try weaker methods of proof.

This example indicates how the discovery of a property (additive identity) can necessitate (or at the least be facilitated by) the coupling of examples and the analysis of examples with the analytic techniques of proofs and explanation based generalization. IL is the first system (that I know of) which is capable of modeling this interplay of empirical techniques and the deductive techniques of theorem proving and EBG as illustrated in this example.

It's worth pointing out that in the Conway Numbers many of the underlying defining sets are infinite, and therefore even the verification of something being an example of a simple property is best treated by a general theorem prover. This is in contrast with AM and Eurisko (Davis & Lenat, 1982; Lenat, 1983) where properties were verified by the execution of Lisp routines associated with those properties.

4 Conclusion

IL is an exploration discovery system that supports an integration of empirical and analytic discovery techniques. The particular domain to which IL is currently applied is Conway numbers, however, there is a (fairly) clean separation of the domain knowledge from the rest of IL and the use of the IL architecture in other domains - even domains outside of mathematics - should be relatively straightforward. We are currently looking at utilizing IL in the discovery of new classes of astronomical objects by examining real astronomical data bases.

The coupling of empirical and analytical discovery methods requires flexible control capabilities and suggests agendas as a reasonable mechanism to employ. Utilizing the "natural" declarative representations used by mathematicians for the concepts, theorems and rules eases the knowledge engineering task, and offer more flexibility and efficiency (I believe) in the use of objects in discovery.

Acknowledgments

My thanks to my thesis advisor, Saul Amarel, for providing guidance and encouragement in this research. Thanks also to Peter Cheeseman, Richard Keller, Don Rosenthal, Lance Albertson and Peter Friedland for suggesting improvements in an earlier draft. This research has been supported in part by the Rutgers Laboratory for Computer Science Research.

References

- Conway, J. H. (1976). *On numbers and games*. New York: Academic Press.
- Davis, R. & Lenat, D. (1982). *Knowledge-based systems in AI*. New York: McGraw-Hill.
- Keller, R. M. (1987). *The role of explicit contextual knowledge in learning concepts to improve performance*. Doctoral dissertation, Department of Computer Science, Rutgers University, New Brunswick, NJ.
- Lakatos, I. (1976). *Proofs and refutations*. Cambridge: Cambridge University Press.
- Lenat, D. (1983). Eurisko: A program that learns new heuristics and domain concepts. *Artificial Intelligence*, 21, 61-98.
- Sims, M. H. (1986). An investigation of the nature of mathematical discovery. In T. M. Mitchell, J. G. Carbonell, & R. S. Michalski (Eds.), *Machine learning: A guide to current research*. Boston, MA: Kluwer Academic.